

AMAZON SHOPVERSE RECORDS

SQL Data Analysis Project

Full Project Documentation

Advanced MySQL Analytics · Business Intelligence · Portfolio Project

A complete end-to-end SQL analytics project covering schema design, data loading, exploration, and 20+ advanced business intelligence queries on e-commerce transaction data using MySQL 8.0+.

This documentation provides a comprehensive technical and business overview of the entire project — suitable for GitHub, interviews, and continuous learning.

Table of Contents

1. Project Overview
2. Objectives and Scope
3. Dataset Description
4. Database Schema Design
5. Project Structure and Repository Layout
6. Setup and Execution Guide
7. Data Exploration and Quality Checks
8. Analytical Queries — Detailed Overview (Queries 1–10)
9. Analytical Queries — Detailed Overview (Queries 11–20)
10. Advanced SQL Techniques Demonstrated
11. Sample Query Logic and Explanation
12. Business Insights Framework
13. Tools and Technology Stack
14. Learning Outcomes and Skills Developed
15. Challenges Faced and Solutions
16. Best Practices Applied
17. Future Enhancements
18. How to Use This Project on GitHub
19. Conclusion
20. Appendix — Quick Reference

1. Project Overview

Amazon Shopverse Records is a professional, end-to-end SQL data analysis project built around a synthetic e-commerce transaction dataset. The project simulates the type of analytical work performed by data analysts and BI developers in real e-commerce organizations.

The core purpose of this project is to demonstrate practical proficiency in MySQL by designing a clean schema, loading data, performing exploration, and answering 20+ realistic business questions using advanced SQL features such as window functions, CTEs, lag calculations, and conditional aggregation.

This project was intentionally structured as a portfolio piece. Analytical questions were designed (with AI assistance) to create a progressive learning path that covers both foundational and advanced SQL concepts while remaining grounded in meaningful business problems — revenue analysis, customer retention, product performance, geographic contribution, operational health, and time-series trends.

The final deliverables include modular SQL scripts, a professional README, a presentation deck, and this full documentation. Together they form a complete package ready for GitHub, LinkedIn, and technical interviews.

1.1 Why This Project Matters

Many SQL tutorials stop at basic SELECT, WHERE, GROUP BY and JOIN. Real-world analytics, however, requires ranking, time-series comparison, retention logic, running totals, and rolling averages. This project deliberately focuses on those advanced patterns so that the skills developed are directly transferable to professional data roles.

1.2 Target Audience

- Aspiring data analysts and SQL developers building a portfolio
- Students preparing for technical interviews that involve SQL case studies
- Professionals transitioning into analytics or business intelligence roles
- Anyone seeking a structured way to practice advanced MySQL

2. Objectives and Scope

2.1 Primary Objectives

- Design a clean, constrained and indexed relational schema suitable for analytical workloads.
- Implement a reliable data-loading process with post-load validation checks.
- Perform thorough exploratory data analysis (EDA) entirely in SQL.
- Solve 20+ business intelligence questions using advanced SQL constructs.
- Organize all work into modular, maintainable scripts suitable for version control.
- Produce professional documentation and a presentation suitable for portfolio use.
- Strengthen practical knowledge of MySQL through real-world analytical scenarios.

2.2 Scope of the Project

The project covers the complete analytics lifecycle inside MySQL: schema design → data ingestion → exploration → advanced analysis. It deliberately excludes external visualization tools (Tableau, Power BI) and Python-based analysis so that the focus remains purely on SQL skill development.

In scope: DDL (CREATE, ALTER, INDEX), DML (LOAD DATA), exploratory queries, window functions, CTEs, time-series logic, ranking, retention analysis, and documentation.

Out of scope: Application development, ETL pipelines outside MySQL, machine learning models, and interactive dashboarding.

2.3 Success Criteria

- All 20 analytical queries execute correctly and produce interpretable results.
- Schema contains proper data types, constraints and useful indexes.
- Repository is clean, modular and ready for public GitHub upload.
- Documentation clearly explains both the technical and business aspects of the work.

3. Dataset Description

The underlying dataset represents order-level transactions from a fictional e-commerce marketplace named Amazon Shopverse. Each row corresponds to a single order line and contains customer, product, financial and geographical attributes. The data spans multiple years (including 2023 and 2024), making time-series and retention analysis possible.

3.1 Nature of the Data

The table is intentionally denormalized (one wide fact table). This is common in analytical projects that start from a flat export. In a production warehouse the same information would typically be normalized into dimension tables (Customer, Product, Date, Seller) and a fact table (Orders). For this portfolio project the single-table design keeps the focus on analytical SQL rather than complex multi-table joins.

3.2 Key Entity Groups

- **Order** — OrderID, OrderDate, OrderStatus, Quantity, TotalAmount
- **Customer** — CustomerID, CustomerName, City, State, Country
- **Product** — ProductID, ProductName, Category, Brand, UnitPrice
- **Financial** — Discount, Tax, ShippingCost, PaymentMethod
- **Seller** — SellerID

3.3 Why This Dataset is Useful for Learning

It contains enough variety (multiple categories, brands, countries, payment methods, order statuses) to support almost every common analytical pattern: aggregation, ranking, cohort-style retention, geographic contribution, discount impact, and rolling averages.

4. Database Schema Design

The schema is defined in `schema.sql`. It creates the database `amazon_shopverse_records` and a single table `shopverse_records` with appropriate data types, a primary key, CHECK constraints and performance indexes.

4.1 Full Column Specification

Column	Type	Constraints / Notes
OrderID	VARCHAR(20)	PRIMARY KEY
OrderDate	DATE	NOT NULL, indexed
CustomerID	VARCHAR(20)	NOT NULL, indexed
CustomerName	VARCHAR(100)	NOT NULL
ProductID	VARCHAR(20)	NOT NULL
ProductName	VARCHAR(100)	NOT NULL
Category	VARCHAR(100)	NOT NULL, indexed
Brand	VARCHAR(100)	NOT NULL, indexed
Quantity	INT	NOT NULL, CHECK > 0
UnitPrice	DECIMAL(12,2)	NOT NULL, CHECK >= 0
Discount	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0
Tax	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0
ShippingCost	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0
TotalAmount	DECIMAL(12,2)	NOT NULL, CHECK >= 0
PaymentMethod	VARCHAR(50)	NOT NULL
OrderStatus	VARCHAR(50)	NOT NULL, indexed
City	VARCHAR(100)	Nullable
State	VARCHAR(100)	Nullable
Country	VARCHAR(100)	NOT NULL, indexed
SellerID	VARCHAR(20)	NOT NULL, indexed

Table 1: Complete column definition of `shopverse_records`

4.2 Design Decisions

- **DECIMAL** is used for all monetary values to avoid floating-point precision issues.
- **CHECK constraints** guard against negative quantities or prices at the database level.
- **Indexes** are placed on columns that appear frequently in WHERE, GROUP BY and ORDER BY clauses of the analytical queries (OrderDate, CustomerID, Category, Brand, Country, SellerID, OrderStatus).
- **Primary key** on OrderID guarantees uniqueness of each order record.

5. Project Structure and Repository Layout

The repository follows a clean, modular layout so that each stage of the analytics lifecycle is isolated in its own script. This makes the project easy to understand, maintain and present.

File / Folder	Purpose
schema.sql	Database creation, table definition, constraints and performance indexes
data_loading.sql	CSV import via LOAD DATA INFILE plus post-load validation queries
exploration.sql	Data profiling: row counts, null checks, sample rows, basic statistics
analysis.sql	All 20+ advanced business intelligence queries (core of the project)
README.md	Project overview, setup instructions and business context for GitHub
results/	Directory for storing exported query outputs and sample findings
.gitignore	Excludes CSV data files and temporary artifacts from version control
docs/	Optional folder for this PDF documentation and presentation files

Table 2: Repository file structure

5.1 Design Philosophy

Each script has a single responsibility. `schema.sql` never contains analysis queries; `analysis.sql` never contains `CREATE TABLE` statements. This separation makes the project easy to navigate for reviewers and for the author when returning to the code after a break.

6. Setup and Execution Guide

6.1 Prerequisites

- MySQL Server 8.0 or higher (required for window functions and CTEs)
- MySQL Workbench, DBeaver, or any SQL client that can execute scripts
- The source CSV file: amazon_shopverse_records.csv
- Sufficient privileges to create databases and load data from file

6.2 Step-by-Step Execution

Step 1 — Create the Schema

Open MySQL and run the entire schema.sql script. This creates the database amazon_shopverse_records, the shopverse_records table, all constraints and indexes. Verify with DESCRIBE shopverse_records;

Step 2 — Prepare and Load Data

Locate the secure-file-priv directory: SHOW VARIABLES LIKE 'secure_file_priv'; Copy the CSV file into that directory. Update the path inside data_loading.sql, then execute the script. The script also runs validation queries that report total rows loaded and the date range of the data.

Step 3 — Explore the Data

Run exploration.sql. Review row counts, null-value checks, sample records and basic descriptive statistics. Confirm that the data looks clean before proceeding to analysis.

Step 4 — Execute Analysis

Run analysis.sql (or execute queries one section at a time). Each query is numbered and commented. Results can be exported to CSV and placed in the results/ folder for the portfolio.

6.3 Common Issues and Fixes

- **LOAD DATA fails** — Check secure_file_priv path and file permissions. Alternatively use the Workbench Import Wizard.
- **Window function errors** — Ensure MySQL version is 8.0+. Older versions do not support OVER().
- **Date format issues** — The loading script uses STR_TO_DATE to convert string dates safely.

7. Data Exploration and Quality Checks

Before running analytical queries it is essential to understand the shape and quality of the data. `exploration.sql` contains a set of profiling queries that answer the following questions:

7.1 Profiling Checklist

- How many total rows (orders) are present?
- How many distinct customers exist?
- What is the earliest and latest OrderDate?
- Are there any NULL values in critical columns (OrderID, OrderDate, TotalAmount, Country, etc.)?
- What is the distribution of OrderStatus values?
- What are the min, max and average TotalAmount?
- How many unique products and categories are present?

7.2 Why Quality Checks Matter

Analytical conclusions are only as good as the underlying data. Nulls in OrderDate would break time-series queries; negative TotalAmount values would distort revenue totals. Running explicit quality checks early prevents silent errors later and demonstrates professional discipline.

8. Analytical Queries — Detailed Overview (Queries 1–10)

This section describes the first ten business questions, the analytical intent behind each, and the main SQL techniques used.

Query 1 — Overall Business KPIs

Calculates total revenue, total orders, average order value and total units sold. This is the classic executive dashboard summary and forms the baseline for all subsequent analysis.

Query 2 — Top 10 Products by Revenue

Ranks products by SUM(TotalAmount). Also returns quantity sold and order count. Helps merchandising teams identify best-selling items.

Query 3 — Monthly Revenue Trends (2023)

Aggregates revenue by year-month for 2023 and sorts by revenue descending. Reveals seasonal patterns and peak months.

Query 4 — Category Performance

Shows total revenue, quantity, average unit price and revenue-share percentage per category. Uses a window function for the share calculation.

Query 5 — Top 5 Customers by Lifetime Value

Ranks customers by total spend. Also returns order count and average order value. Supports CRM and loyalty program decisions.

Query 6 — Geographic Revenue Contribution

Filters to Delivered orders, groups by Country, and computes each country's share of total revenue using a window function. Useful for market prioritization.

Query 7 — Discount Analysis

Estimates total discount given and average discount percentage overall and by Brand. Helps evaluate promotional effectiveness.

Query 8 — Payment Method Performance

Counts orders and sums revenue by PaymentMethod. Sorted by revenue to show preferred payment channels.

Query 9 — Product Ranking within Categories

Uses a CTE and DENSE_RANK() partitioned by Category to surface the top 3 products inside each category. Classic window-function ranking pattern.

Query 10 — High-Value US Delivered Orders (2024)

Multi-condition filter: year = 2024, status = Delivered, TotalAmount > 1000, Country = United States. Returns detailed order lines for further investigation.

9. Analytical Queries — Detailed Overview (Queries 11–20)

Query 11 — Year-over-Year Revenue Growth

Builds a yearly revenue CTE, then uses LAG() to compute the percentage change from the previous year. Essential for growth reporting.

Query 12 — Customer Retention (2023 vs 2024)

Uses conditional aggregation (CASE WHEN) to split revenue by year, then keeps only customers who appear in both years (HAVING COUNT(DISTINCT year) = 2). Measures true retention.

Query 13 — Peak Sales by Day of Week

Applies DAYNAME(OrderDate) and calculates average order value and order count per weekday. Supports staffing and marketing calendar decisions.

Query 14 — Return / Cancellation Rate by Category

Flags orders whose status is Returned or Cancelled and computes the percentage of such “bad” orders per category. Highlights quality or expectation problems.

Query 15 — Top Sellers Performance

Ranks SellerID by total revenue and also shows average order value and order count. Useful for marketplace seller management.

Query 16 — Brand Price Range Analysis

Computes MIN, MAX, AVG UnitPrice and the price range (Max – Min) for each Brand. Reveals pricing strategy differences across brands.

Query 17 — Running Cumulative Revenue

Groups by year-month and uses SUM() OVER (ORDER BY year-month) to produce a running total. Shows how revenue accumulates over time.

Query 18 — Multi-Country Comparison

Restricts to United States, India and Australia; compares total revenue, average shipping cost and average order value. Supports international expansion analysis.

Query 19 — High Quantity High-Value Orders

Filters Quantity >= 5 AND TotalAmount > 1500. Surfaces bulk or high-ticket purchases.

Query 20 — 3-Month Rolling Average Revenue

Builds a monthly revenue CTE, then applies AVG() OVER with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW to smooth monthly noise and reveal underlying trends.

10. Advanced SQL Techniques Demonstrated

The project deliberately exercises a wide range of intermediate and advanced MySQL features. Below is a summary of the techniques and where they appear.

10.1 Window Functions

Window functions compute values across a set of rows related to the current row without collapsing the result set the way GROUP BY does. In this project they are used for ranking (DENSE_RANK), year-over-year comparison (LAG), running totals (SUM OVER ORDER BY), revenue share percentages, and rolling averages (AVG OVER ROWS BETWEEN).

10.2 Common Table Expressions (CTEs)

CTEs improve readability by breaking complex logic into named intermediate steps. They appear in the product-ranking query, the YoY growth query and the rolling-average query. Using CTEs also makes debugging easier because each step can be inspected independently.

10.3 Conditional Aggregation

CASE WHEN expressions inside SUM or COUNT allow multiple metrics to be calculated in a single pass. This pattern is used for customer retention (splitting revenue by year) and for return-rate calculation (counting bad orders).

10.4 Time-Series Patterns

Functions such as YEAR(), DATE_FORMAT(), DAYNAME() combined with grouping and window logic enable monthly trends, day-of-week analysis, YoY growth and rolling averages — all essential for e-commerce reporting.

10.5 Indexing Strategy

Indexes were added on columns that appear repeatedly in filters and grouping clauses. This reduces full-table scans and demonstrates awareness of query performance even on a modest-sized dataset.

11. Sample Query Logic and Explanation

To illustrate the analytical thinking behind the code, two representative queries are explained in detail.

11.1 Example — Year-over-Year Growth (Query 11)

First a CTE aggregates total revenue by calendar year. Then the outer query uses `LAG(total_revenue) OVER (ORDER BY sales_year)` to bring the previous year's revenue onto the same row. The growth percentage is calculated as $(\text{current} - \text{previous}) / \text{previous} \times 100$. The first year naturally returns NULL for growth because there is no prior year.

This pattern is foundational for any growth or trend report and appears in almost every BI tool.

11.2 Example — Customer Retention (Query 12)

Conditional aggregation creates two columns: `revenue_2023` and `revenue_2024`. The `HAVING` clause `COUNT(DISTINCT YEAR(OrderDate)) = 2` keeps only customers who placed at least one order in each year. The result set therefore represents true retained customers rather than simple year-over-year totals.

Retention analysis is one of the highest-value analyses in e-commerce and SaaS; mastering this pattern is directly useful in professional roles.

11.3 Example — 3-Month Rolling Average (Query 20)

A monthly CTE is created first. The outer query then applies `AVG(total_revenue) OVER (ORDER BY year_month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`. This frame includes the current month and the two preceding months, producing a smoothed trend line that reduces monthly noise.

12. Business Insights Framework

Beyond writing correct SQL, a strong analyst must interpret results in business terms. The queries in this project map to the following decision areas:

12.1 Revenue & Growth

Queries 1, 3, 11, 17 and 20 answer: How much are we making? Is revenue growing? Which months drive the business? What is the underlying trend after smoothing?

12.2 Product & Category Strategy

Queries 2, 4, 9 and 16 answer: Which products and categories generate the most revenue? Where are the price-range opportunities? Which items lead inside each category?

12.3 Customer Intelligence

Queries 5 and 12 answer: Who are our highest-value customers? Which customers returned year after year? These insights feed loyalty programs and personalised marketing.

12.4 Operations & Risk

Queries 7, 8, 13, 14 and 15 answer: How aggressive are discounts? Which payment methods dominate? Which weekdays are busiest? Where are return rates highest? Who are the top sellers?

12.5 Geographic Expansion

Queries 6 and 18 answer: Which countries contribute most to delivered revenue? How do shipping costs and average order values compare across key markets?

13. Tools and Technology Stack

Tool / Technology	Role in the Project
MySQL 8.0+	Core database engine; provides window functions, CTEs, DATE functions
MySQL Workbench	Primary IDE for writing, testing and executing SQL scripts
SQL (DDL + DML + Analytics)	Schema design, data loading, exploration and all analytical queries
Git / GitHub	Version control and public portfolio presentation
Markdown (README.md)	Human-readable project documentation on GitHub
CSV	Source data format loaded via LOAD DATA INFILE
PowerPoint / PDF	Presentation deck and this full documentation deliverable

Table 3: Technology stack

13.1 Why MySQL 8.0+

Window functions and CTEs became fully usable in MySQL 8.0. Older versions would force work-arounds with user variables or temporary tables, making the code harder to read and less portable. Targeting MySQL 8.0+ keeps the SQL modern and aligned with current industry practice.

14. Learning Outcomes and Skills Developed

Completing this project produced measurable growth in practical SQL ability:

14.1 Technical Skills

- Writing production-style analytical queries rather than textbook examples
- Confident use of window functions (RANK/DENSE_RANK, LAG, running totals, rolling averages)
- Designing readable multi-step logic with CTEs
- Applying conditional aggregation for retention and status-based metrics
- Creating indexes that support analytical filter and grouping patterns
- Performing systematic data quality and profiling checks in SQL

14.2 Analytical / Business Skills

- Translating open-ended business questions into precise SQL logic
- Interpreting query results in terms of revenue, retention, product strategy and operations
- Structuring an analysis so that each query answers a clear decision need

14.3 Professional / Portfolio Skills

- Organising work into modular scripts with single responsibilities
- Writing clear documentation and README material for public review
- Packaging a complete project (code + docs + presentation) for GitHub and interviews

15. Challenges Faced and Solutions

15.1 Data Loading Path Issues

MySQL's `secure_file_priv` setting restricts where `LOAD DATA INFILE` may read files. Solution: always check the variable first, place the CSV in the allowed directory, and document the path clearly in `data_loading.sql`. As a fallback, the Workbench Import Wizard can be used.

15.2 Date Format Variability

CSV date strings may not match MySQL's default date format. Solution: load the date column into a user variable and apply `STR_TO_DATE` with an explicit format mask.

15.3 Window Function Learning Curve

Understanding `PARTITION BY` versus `ORDER BY` frames, and the difference between `ROWS` and `RANGE`, takes practice. Solution: start with simple ranking, then add `LAG`, then running totals, and finally rolling averages — exactly the progression used in this project.

15.4 Keeping Scripts Maintainable

A single giant SQL file becomes hard to navigate. Solution: strict separation into schema, loading, exploration and analysis scripts, each with clear section comments and consistent naming.

16. Best Practices Applied

Clear Naming — Tables, columns and aliases use descriptive names (TotalAmount, year_month, rank_in_category) instead of cryptic abbreviations.

Section Comments — Every major query block is preceded by a numbered comment header so the file is self-documenting.

Consistent Rounding — Monetary results are rounded to two decimal places with ROUND() for readability.

Explicit Ordering — Results that are ranked or time-based always include an ORDER BY clause.

Defensive Coding — NULLIF is used when dividing by a previous-year value that could be zero; CHECK constraints prevent invalid data at insert time.

Index Awareness — Indexes are created for columns that appear in filters and groupings of the analytical queries.

Modular Files — DDL, loading, exploration and analysis live in separate files with single responsibilities.

Portfolio Mindset — Every artefact (README, PDF, presentation) is written so that a third-party reviewer can understand the work without additional explanation.

17. Future Enhancements

The current project is complete as a pure-SQL portfolio piece. Possible next steps include:

17.1 Technical Extensions

- Create reusable VIEWS for the most common KPI queries
- Write stored procedures with date-range parameters for flexible reporting
- Introduce a proper star-schema (fact + dimensions) for larger-scale BI work
- Add unit tests for critical queries using a SQL testing framework

17.2 Visualisation & Reporting

- Connect the same queries to Tableau or Power BI for interactive dashboards
- Use Python (Pandas + SQLAlchemy + Matplotlib/Plotly) for automated visual reports
- Generate scheduled PDF or email summaries of key KPIs

17.3 Data Model Evolution

- Split the denormalized table into Customer, Product, Date, Seller dimensions and an Orders fact table
- Add slowly-changing-dimension logic for product and customer attributes

18. How to Use This Project on GitHub

18.1 Recommended Repository Layout

Place the SQL scripts at the root, put this PDF and the presentation inside a docs/ folder, and keep a results/ folder for any exported CSVs. A strong README.md at the root should summarise the project, list the tech stack, and give one-command setup instructions.

18.2 Suggested README Sections

- Project title and one-paragraph description
- Key features / analytical coverage
- Tech stack
- Project structure (tree view)
- Setup & execution steps
- Sample insights or screenshots of query results
- Learning outcomes
- License (MIT is a common choice for portfolio work)

18.3 Topics / Tags for GitHub

sql, mysql, data-analysis, business-intelligence, data-analytics, ecommerce, window-functions, cte, sql-project, portfolio, data-exploration, revenue-analysis, customer-analytics

19. Conclusion

Amazon Shopverse Records is a complete, professional SQL data analysis project that walks through the full analytics lifecycle inside MySQL — from schema design and data loading, through exploration, to twenty advanced business intelligence queries. The work demonstrates both technical depth (window functions, CTEs, time-series logic, indexing) and analytical thinking (revenue, retention, product strategy, operations, geography).

By structuring the questions progressively and packaging the results as modular scripts, a clear README, a presentation and this documentation, the project becomes a strong portfolio asset. It shows not only that the author can write SQL, but that the author can organise, document and communicate analytical work at a professional standard.

Most importantly, the project was built with a deliberate learning goal: to grow practical knowledge of the SQL language using MySQL through realistic e-commerce scenarios. That goal has been met, and the same pattern can be repeated with new datasets to continue improving.

20. Appendix — Quick Reference

20.1 Query Index (Quick Lookup)

#	Query Focus	Key Technique
1	Overall KPIs	Aggregation
2	Top Products	GROUP BY + ORDER BY
3	Monthly Trends 2023	DATE_FORMAT
4	Category Performance	Window % share
5	Top Customers (LTV)	GROUP BY ranking
6	Country Contribution	Window % share
7	Discount Analysis	Brand-level AVG
8	Payment Methods	GROUP BY
9	Rank within Category	DENSE_RANK + CTE
10	High-Value US 2024	Multi-condition filter
11	YoY Growth	LAG + CTE
12	Customer Retention	CASE + HAVING
13	Peak Day of Week	DAYNAME
14	Return Rate by Category	Conditional COUNT
15	Top Sellers	GROUP BY ranking
16	Brand Price Range	MIN/MAX/AVG
17	Cumulative Revenue	SUM OVER ORDER BY
18	Country Comparison	Filtered GROUP BY
19	High Qty + High Value	Compound WHERE
20	3-Month Rolling Avg	AVG OVER ROWS

Table 4: Quick reference of all 20 analytical queries

20.2 Key MySQL Functions Used

SUM, COUNT, AVG, MIN, MAX, ROUND, YEAR, DATE_FORMAT, DAYNAME, DENSE_RANK, LAG, SUM() OVER, AVG() OVER, CASE WHEN, COALESCE / NULLIF, STR_TO_DATE, LOAD DATA INFILE

This project was developed to strengthen practical knowledge of the SQL language using MySQL. Analytical questions were structured with AI support to create a progressive and professional learning path. All scripts, documentation and presentation materials are intended for portfolio and educational use.